

Data Science

COMP5122M

Exploratory Data Analysis
and
Visualization

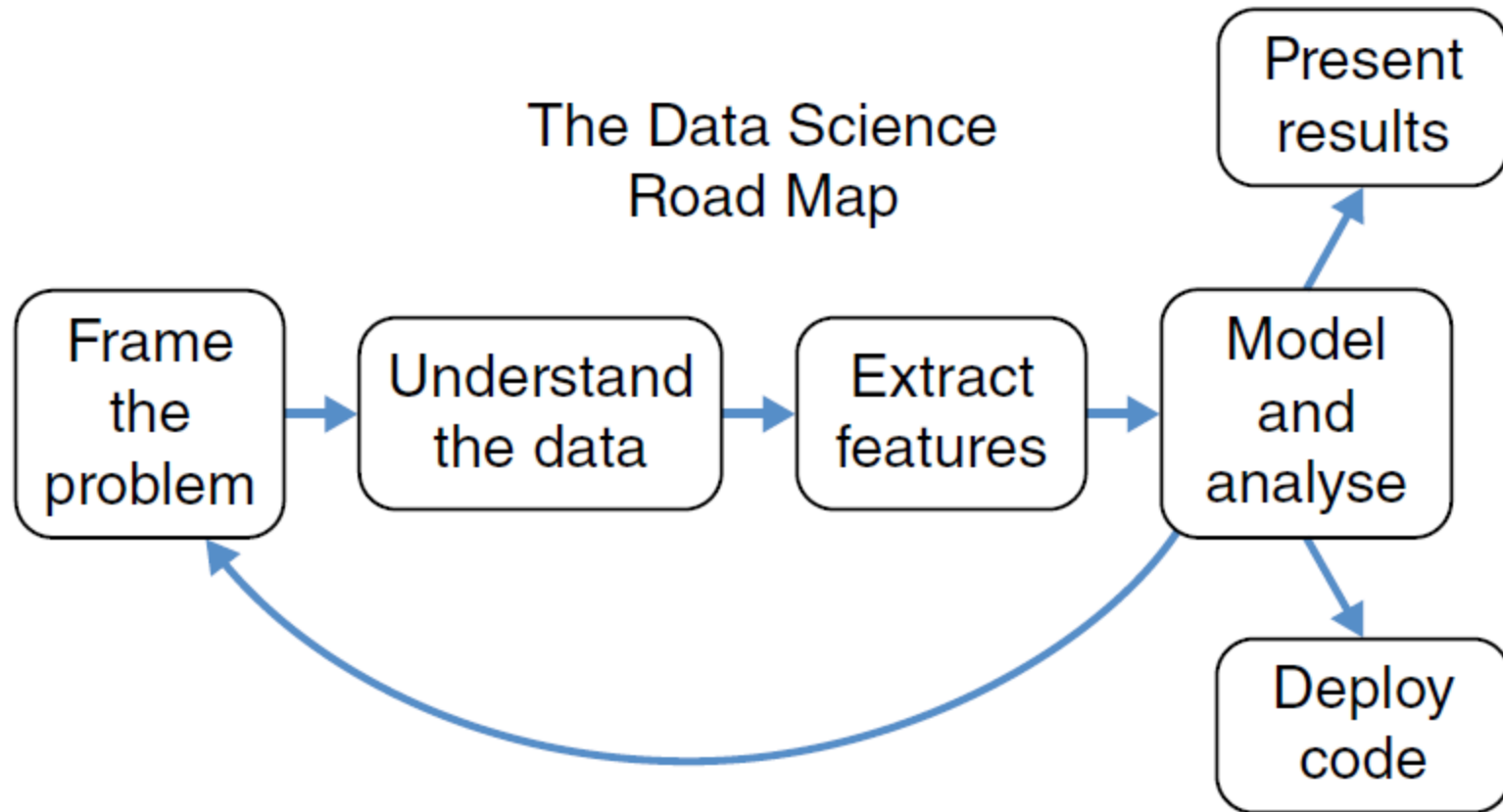
Duygu Sarikaya

School of Computing, University of Leeds

What is Data Science?

The purpose of computing is insight, not numbers.
– Richard W. Hamming

Data Science Road Map





EDA is unboxing for data!

Understand the Data

Exploratory Data Analysis (EDA) Guiding Principles

“Exploratory data analysis is an attitude, a state of flexibility, a willingness to look for those things that we believe are not there, as well as those that we believe to be there.”

– John Tukey

Example

Objective: Predict the survival of the passengers aboard RMS Titanic.

Dataset: Titanic DataSet

- provides the data on all the passengers who were aboard the RMS Titanic when it sank on 15 April 1912 after colliding with an iceberg in the North Atlantic ocean.
 - a combination of variables based on personal characteristics such as age, class of ticket and gender, and tests one's classification skills.
 - 891 rows and 12 columns



What is Data?

E.F.O. 13006 Ocean Ticket No. 347077

WHITE STAR LINE.
OCEANIC STEAM NAVIGATION CO. LIMITED.

EMIGRANT INLAND FORWARDING ORDER.

**TO WHITE STAR LINE,
NEW YORK.**

This will entitle the bearer Charles A. Phelan
to Emigrant Class passage for Seven persons making Four Adults,
from **NEW YORK** to Worcester, Mass.
which charge to account of S.S. Titanic sailing 10 APR 1912
Amount of Inland Fare, £ 2-16-4
For ISMAY, IMRIE & CO.,
Date of Booking 16 APR 1912 19
19

WHITE STAR LINE
ROYAL AND UNITED STATES MAIL STEAMERS.

ISMAY, IMRIE & CO.,
1, DOCKSPUR STREET, S.W.,
88, LEADENHALL STREET, E.C.,
LONDON,
80, JAMES STREET,
LIVERPOOL,
AND
CANUTE ROAD, SOUTHAMPTON.

Agent at PARIS—
NICHOLAS MARTIN, 9, Rue Scribe.

WHITE STAR LINE,
10, VIA ALLA NUZZIATA, GENOA,
21, PIAZZA DELLA BORSA, NAPLES,
64, STATE STREET, BOSTON,
9, BROADWAY, NEW YORK,
83, DALHOUSIE STREET, QUEBEC,
BELL TELEPHONE BUILDINGS,
118, NOTRE DAME STREET WEST, MONTREAL.

**JAMES SCOTT & CO., Agents,
QUEENSTOWN.**

OCEANIC STEAM NAVIGATION COMPANY, LIMITED, OF GREAT BRITAIN.
THIRD CLASS (Steerage) PASSENGER'S CONTRACT TICKET.
(NOT TRANSFERABLE).

SHIP R.M.S. TITANIC of 45,000 Tons Register,
to take in Passengers at QUEENSTOWN for NEW YORK
on the 11 day of APRIL 191912

NAMES.	AGE.	No. of Bunks Allocated.
<u>Dorothy Ryan</u>	<u>32</u>	<u>3</u>

Breakfast at Eight O'clock. — Oatmeal Porridge and Milk, Tea, Coffee, Sugar, Milk, Fresh Bread and Butter, Herrings, Potatoes, Ling Fish and Egg Sauce, or Irish Stew, according to the day of the week.
Dinner at One O'clock. — Soup, Beef, Mutton, Carrots and Turnips, Green Peas, Pork or Ling Fish and Sauce with Bread and Potatoes, Plum Pudding, Rice Pudding, Stewed Apples and Rice, or Stewed Prunes and Rice, according to the day of the week.
Tea at Six O'clock. — Tea, Sugar, Milk, Fresh Bread and Butter, Jam, or Cheese and Pickles, Corned Beef, Hash, or Timed Beef, according to the day of the week. Oatmeal Gruel will be supplied at 8 p.m.
NOTE.—For Women and Children—Chicken Broth or Beef Tea Daily at 11 a.m., and Tea, Coffee and Milk at all hours.
Mess. Utensils and Bedding provided by the Ship.
For and on behalf of the OCEANIC STEAM NAVIGATION COMPANY, LIMITED, OF GREAT BRITAIN.

NOTICE TO STEERAGE PASSENGERS.—1.—If Steerage Passengers, through no default of their own, are not received on board on the day named in their contract tickets, or fail to obtain a Passage in the Ship, they should apply to the Emigration Officer at the port, who will assist in obtaining redress, under the Merchant Shipping Act.—2.—Third Class (Steerage) Passengers should carefully keep this part of their Contract Ticket till the end of the Voyage. This Contract Ticket is exempt from Stamp duty.

 **WHITE STAR LINE**

YOUR ATTENTION IS SPECIALLY DIRECTED TO THE CONDITIONS OF
TRANSPORTATION IN THE ENCLOSED CONTRACT.

THE COMPANY'S LIABILITY FOR BAGGAGE IS STRICTLY LIMITED, BUT
PASSENGERS CAN PROTECT THEMSELVES BY INSURANCE.

First-Class Passenger Ticket per Steamship Titanic
SAILING FROM 10/4 1912

Visualizations and Simple Metrics

Typically, a good analytics project starts (after cleaning and understanding the data) with exploratory visualizations that help you develop hypotheses and get a feel for the data.

- Then you carefully manicure figures that make the final results visually obvious.

Today:

- We will cover several of the most important visualization techniques.
- We will also cover some exploratory metrics (such as correlations), which capture some of the patterns that are clear from a good visual
- Our eyes can trick us, so it's important to have a cold hard statistic too.
 - We will cover some standard statistical metrics

matplotlib

The main visualization tool for Python is a library called matplotlib.

- the most standard tool
- sufficient for most data science
- integrates well with the other libraries
- powerful, flexible

vs.

- Syntax is not intuitive
- Interactivity



pandas

Pandas is a library written for the Python programming language for data manipulation and analysis.

- Offers data structures and operations for manipulating numerical tables and time series.
- Visualizations
 - wrapper-around matplotlib

Typically, you make an image by calling the `plot()` method on a Pandas object, and Pandas does all the image formatting under the hood.

Then you use matplotlib's `pyplot` module for things such as setting the title and the final act of either displaying the image or saving it to a file.



Numpy

Numpy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays.

- Highly optimized library use for numerical computation



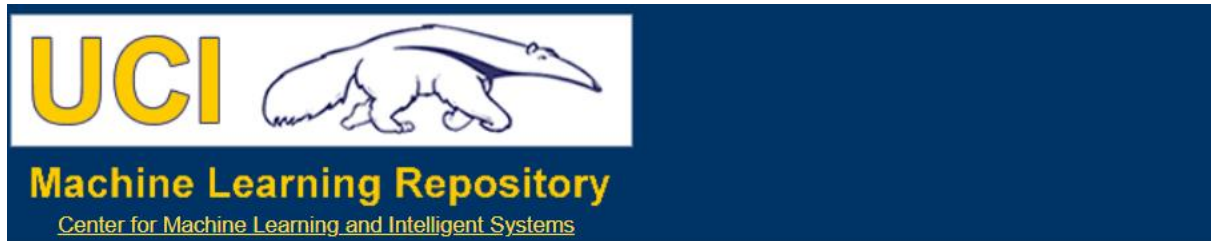
Scikit-learn

Scikit-learn is a free software machine learning library for the Python programming language

- features various classification, regression and clustering algorithms including support vector machines, random forests, gradient boosting, k-means and DBSCAN,
- designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.



Iris Dataset



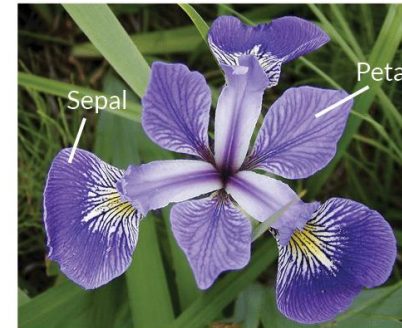
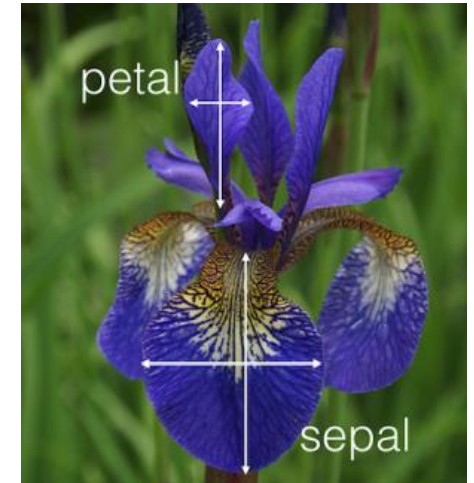
Iris Data Set

Download: [Data Folder](#), [Data Set Description](#)

Abstract: Famous database; from Fisher, 1936



Data Set Characteristics:	Multivariate	Number of Instances:	150	Area:	Life
Attribute Characteristics:	Real	Number of Attributes:	4	Date Donated	1988-07-01
Associated Tasks:	Classification	Missing Values?	No	Number of Web Hits:	3618679



Iris Versicolor



Iris Setosa



Iris Virginica

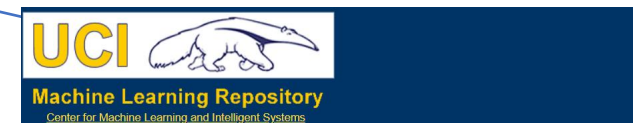
An example

Import the relevant libraries and create a DataFrame containing the sample dataset (which comes built-in to scikit-learn):

```
import pandas as pd
from matplotlib import pyplot as plt
import sklearn.datasets

def get_iris_df():
    ds = sklearn.datasets.load_iris()
    df = pd.DataFrame(ds['data'],
                      columns = ds['feature_names'])
    code_species_map = dict(zip(
        range(3), ds['target_names']))
    df['species'] = [code_species_map[c]
                    for c in ds['target']]
    return df

df = get_iris_df()
```



Iris Data Set
Download [Data Folder](#) [Data Set Description](#)
Abstract: Famous database; from Fisher, 1936



Data Set Characteristics:	Multivariate	Number of Instances:	150	Area:	Life
Attribute Characteristics:	Real	Number of Attributes:	4	Date Donated	1988-07-01
Associated Tasks:	Classification	Missing Values?	No	Number of Web Hits:	3618679

An example

Import the relevant libraries and create a DataFrame containing the sample dataset (which comes built-in to scikit-learn):

```
import pandas as pd
from matplotlib import pyplot as plt
import sklearn.datasets

def get_iris_df():
    ds = sklearn.datasets.load_iris()
    df = pd.DataFrame(ds['data'],
                      columns = ds['feature_names'])
    code_species_map = dict(zip(
        range(3), ds['target_names']))
    df['species'] = [code_species_map[c]
                    for c in ds['target']]
    return df

df = get_iris_df()
```

Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	Iris setosa
4.9	3.0	1.4	0.2	Iris setosa
7.0	3.2	4.7	1.4	Iris versicolor
6.4	3.2	4.5	1.5	Iris versicolor
6.3	3.3	6.0	2.5	Iris virginica
5.8	3.3	6.0	2.5	Iris virginica

But first, some data exploration

#Let's break it down: Load Iris dataset



```
from sklearn.datasets import load_iris  
iris = load_iris()
```



Machine Learning Repository
Center for Machine Learning and Intelligent Systems

Iris Data Set
Download [Data Folder](#) [Data Set Description](#)

Abstract: Famous database; from Fisher, 1936



Data Set Characteristics:	Multivariate	Number of Instances:	150	Area:	Life
Attribute Characteristics:	Real	Number of Attributes:	4	Date Donated	1988-07-01
Associated Tasks:	Classification	Missing Values?	No	Number of Web Hits:	3618679

But first, some data exploration

#Let's break it down: Now we have loaded iris dataset let's print its features/input and target/output

Store features matrix in X

X= iris.data

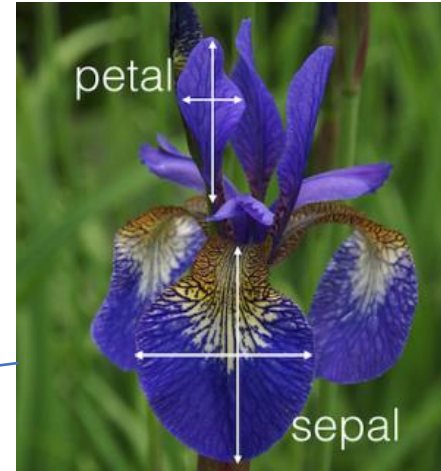
#Store target (labels) vector in

y= iris.target

Names of features/columns in iris dataset

print(iris.feature_names)

Output : ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']



we have 4 features in the iris dataset named Sepal, Petal, length, and width.

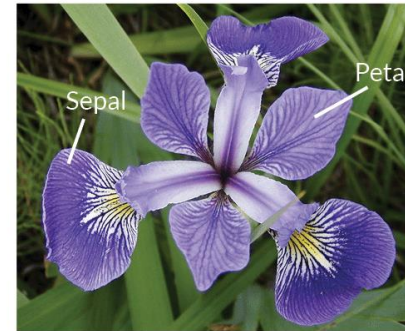
But first, some data exploration

#Let's break it down: Now we have loaded iris dataset let's print its features/input and target/output

Names of target/output in iris dataset

```
print(iris.target_names)
```

Output : ['setosa' 'versicolor' 'virginica']



Iris Versicolor



Iris Setosa



Iris Virginica

But first, some data exploration

#Let's break it down: Dimensions/Size of Iris dataset

size of feature matrix

```
print(iris.data.shape)
```

Output: (150, 4)

#we have total of 150 rows/samples/observations/records/instances and 4 features/columns

But first, some data exploration

```
# size of target vector  
print(iris.target.shape)  
Output : (150,)
```

Iris target stores 150 samples of iris species

But first, some data exploration

#how is it stored?

Feature Data

#let's print features

```
print(iris.data)
```

Output

```
[[5.1      3.5      1.4      0.2]  
 [4.9      3.      1.4      0.2]  
 [4.7      3.2      1.3      0.2]  
 [4.6      3.1      1.5      0.2]  
 [5.       3.6      1.4      0.2] ...}
```

#5.1 is sepal length (cm), 3.5 is sepal width (cm), ... etc

But first, some data exploration

#how is it stored?

Target Data

```
# print target vector iris species: 0 = setosa, 1 = versicolor, 2 = virginica
```

```
print(iris.target)
```

Output :

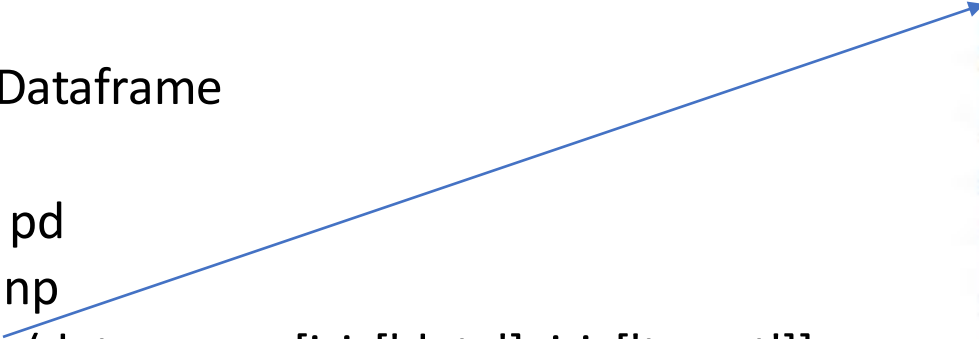
[illegible]

But first, some data exploration

#This built-in iris dataset is stored as a NumPy array so first let's convert it into a data frame using pandas

#Converting into Dataframe

```
import pandas as pd
import numpy as np
df = pd.DataFrame(data= np.c_[iris['data'], iris['target']],
                  columns= iris['feature_names'] + ['Species'])
```



Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	Iris setosa
4.9	3.0	1.4	0.2	Iris setosa
7.0	3.2	4.7	1.4	Iris versicolor
6.4	3.2	4.5	1.5	Iris versicolor
6.3	3.3	6.0	2.5	Iris virginica
5.8	3.3	6.0	2.5	Iris virginica

But first, some data exploration

```
#Get Distribution of Iris species in the target variable
```

```
df['Species'].value_counts()
```

Output :

```
2.0    50
```

```
1.0    50
```

```
0.0    50
```

```
Name: Species, dtype: int64
```

```
# Here we can see the distribution of each Iris species is equal (50 samples for each class
```

```
# where the total of 3 classes is 150 samples
```

But first, some data exploration

#Describe function

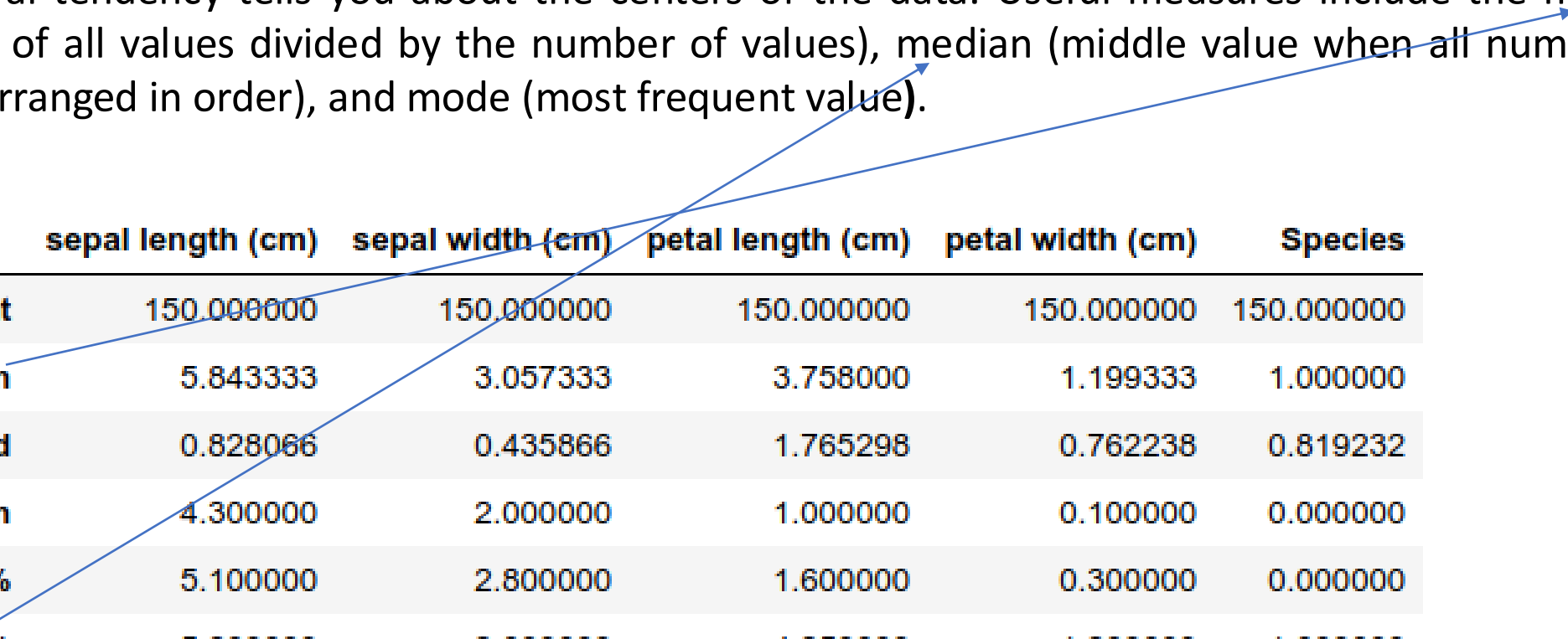
Using the describe function give us statistical information about our dataset

df.describe()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

But first, some data exploration

Central tendency tells you about the centers of the data. Useful measures include the mean (sum of all values divided by the number of values), median (middle value when all numbers are arranged in order), and mode (most frequent value).



	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

But first, some data exploration

Variability tells you about the spread of the data. Useful measures include variance (how far each value in the dataset is from the mean, on average) and standard deviation (the square root of the variance).

Standard Deviation of the Values



	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

An example

Import the relevant libraries and create a DataFrame containing the sample dataset (which comes built-in to scikit-learn):

```
import pandas as pd
from matplotlib import pyplot as plt
import sklearn.datasets

def get_iris_df():
    ds = sklearn.datasets.load_iris()
    df = pd.DataFrame(ds['data'],
                      columns = ds['feature_names'])
    code_species_map = dict(zip(
        range(3), ds['target_names']))
    df['species'] = [code_species_map[c]
                    for c in ds['target']]
    return df

df = get_iris_df()
```

Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	Iris setosa
4.9	3.0	1.4	0.2	Iris setosa
7.0	3.2	4.7	1.4	Iris versicolor
6.4	3.2	4.5	1.5	Iris versicolor
6.3	3.3	6.0	2.5	Iris virginica
5.8	3.3	6.0	2.5	Iris virginica

Pie Charts

One of the clearest ways to present data:

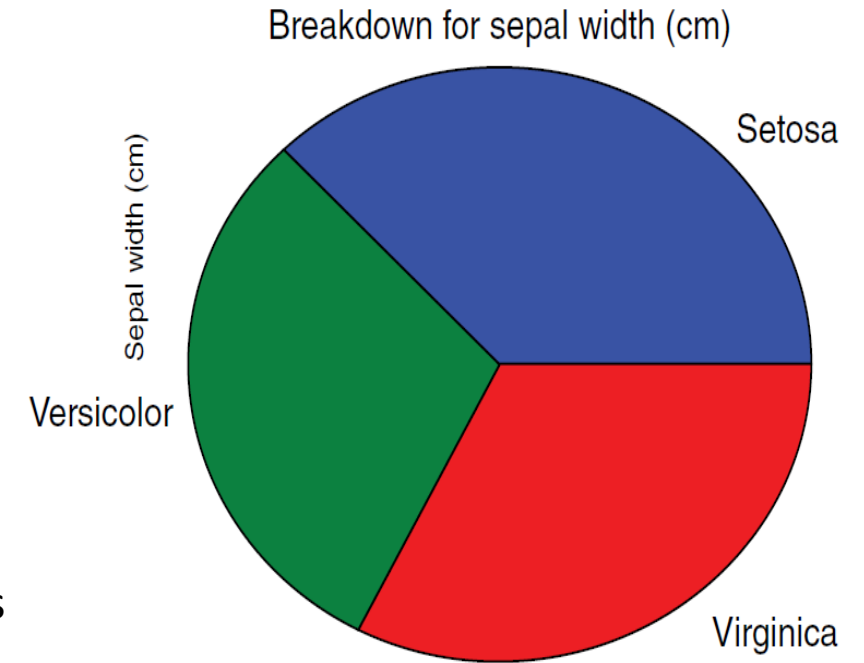
- making sense of the numbers requires cognitive effort and attention.
 - on the other hand, we make immediate sense of pie charts with less mental effort.
 - conveys information in a way that the human brain will understand and care about.
-
- Where to use?
 - doing exploratory analysis of a dataset (How many of our customers are senior citizens? How many of the page views came from Leeds?)
 - communicating the results of a classifier.

Pie Charts

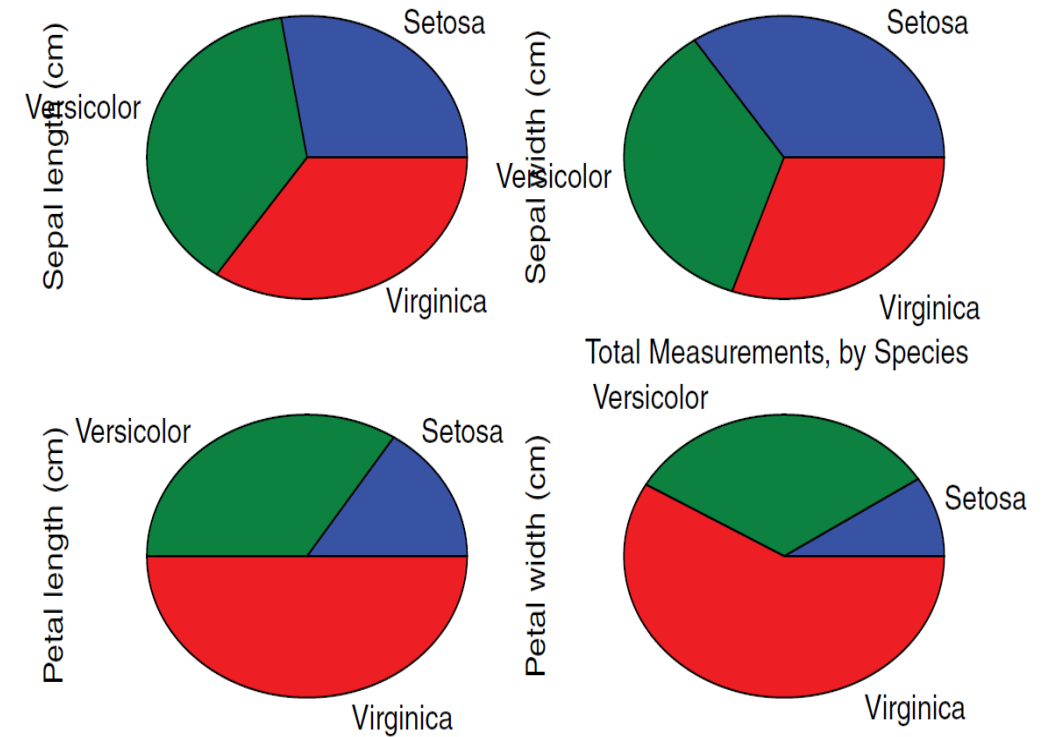
#Generating a basic pie chart :

#calling the plot() method on a Pandas Series object whose index gives us
#the flower species

```
sums_by_species = df.groupby('species').sum()  
var = 'sepal width (cm)'  
sums_by_species[var].plot(kind='pie', fontsize=20)  
plt.ylabel(var, horizontalalignment='left')  
plt.title('Breakdown for ' + var, fontsize=25)  
plt.savefig('iris_pie_for_one_variable.jpg')  
plt.close()
```



Pie Charts



#Generating a basic pie chart :

#calling plot() on a DataFrame with multiple columns, we can generate a different chart for each column all in
#the same figure

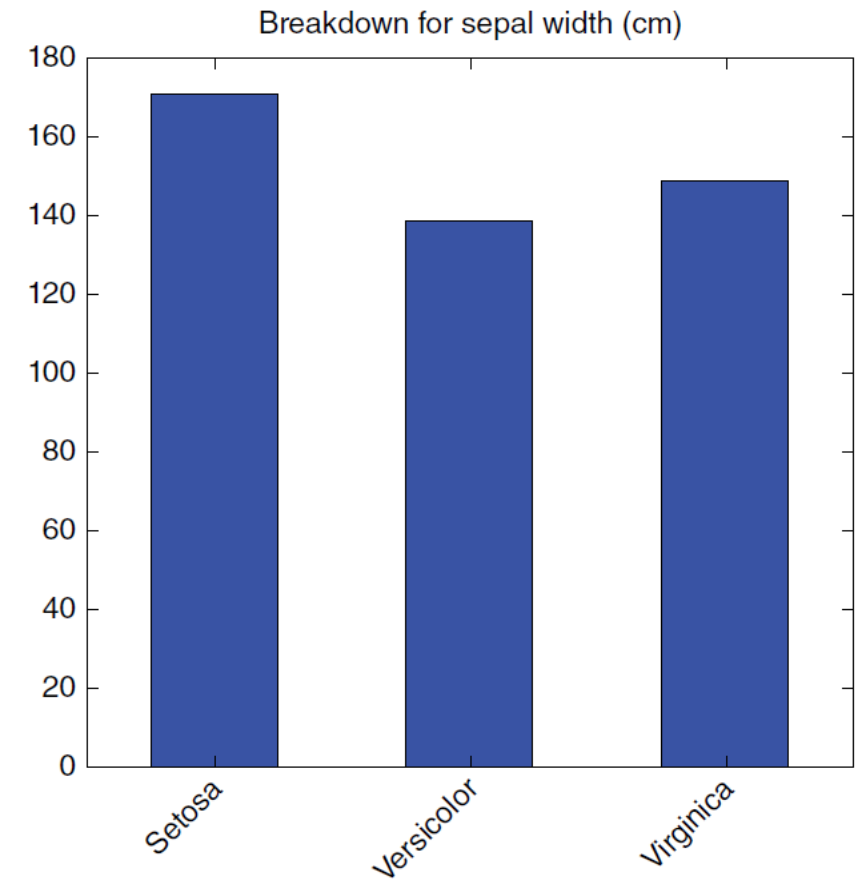
```
sums_by_species = df.groupby('species').sum()
sums_by_species.plot(kind='pie', subplots=True,
layout=(2,2), legend=False)
plt.title('Total Measurements, by Species')
plt.savefig('iris_pie_for_each_variable.jpg')
plt.close()
```

Bar Charts

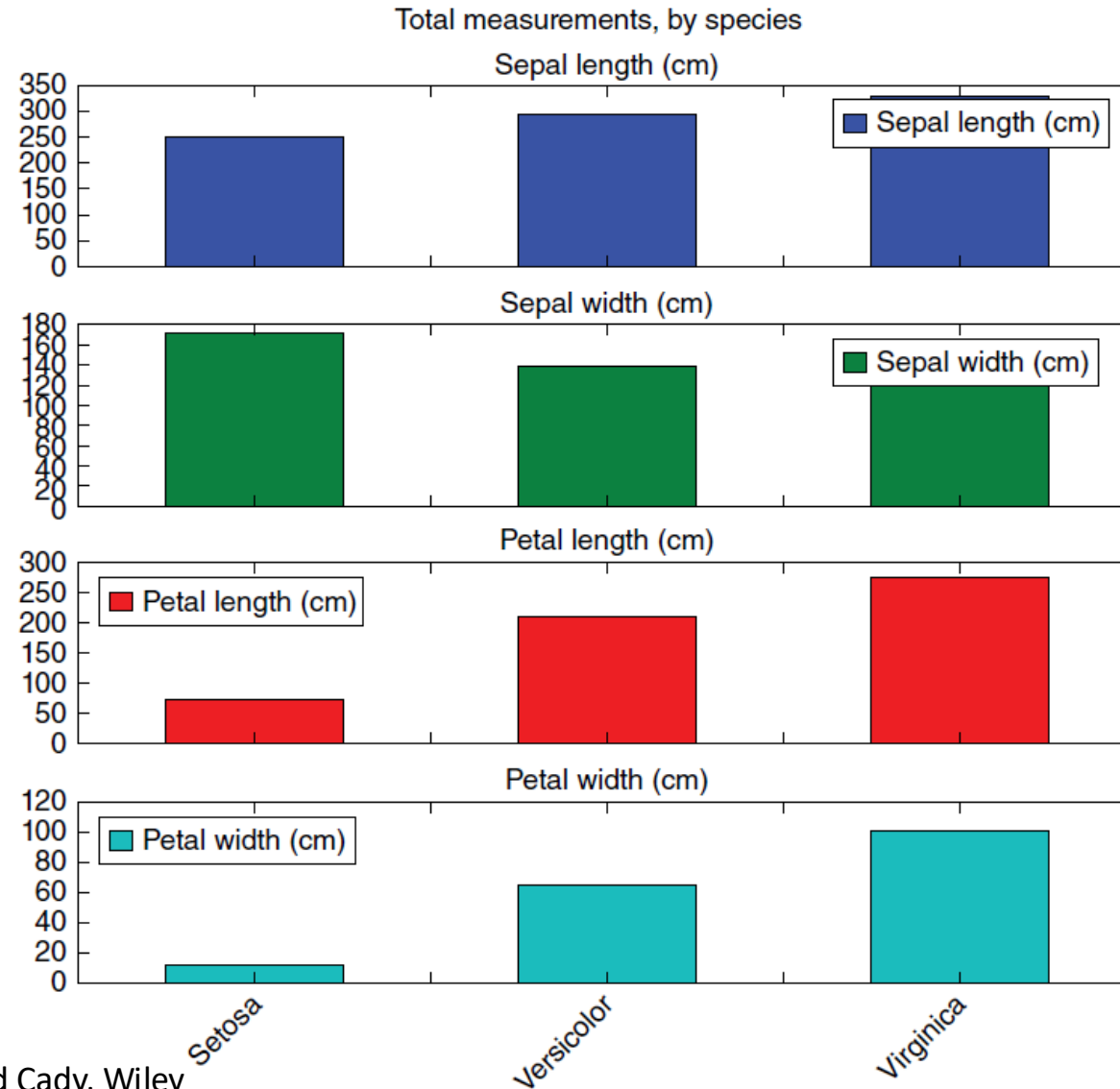
#Generating a basic bar chart :

#relative sizes of the different flowers

```
sums_by_species = df.groupby('species').sum()
var = 'sepal width (cm)'
sums_by_species[var].plot(kind='bar', fontsize=15,
rot=30)
plt.title('Breakdown for ' + var, fontsize=20)
plt.savefig('iris_bar_for_one_variable.jpg')
plt.close()
sums_by_species = df.groupby('species').sum()
sums_by_species.plot(
    kind='bar', subplots=True, fontsize=12)
plt.suptitle('Total Measurements, by Species')
plt.savefig('iris_bar_for_each_variable.jpg')
plt.close()
```



Bar Charts

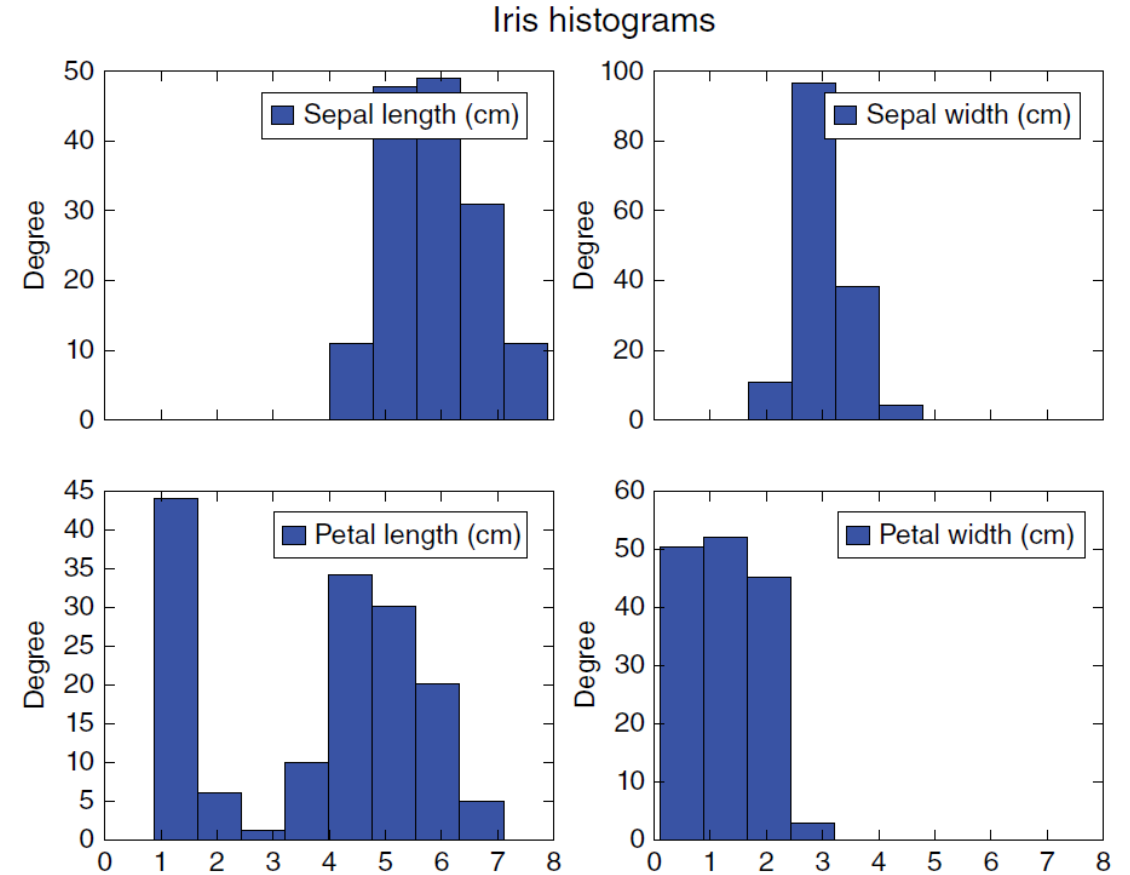


Histograms

- a sense of whether there are a few distinct outliers
- how much variation is in the population etc.

#Generating histograms for all the columns and put them together in one figure :

```
df.plot(kind='hist', subplots=True, layout=(2,2))
plt.suptitle('Iris Histograms', fontsize=20)
plt.show()
```



- petal length has a clearly bimodal distribution, suggesting that this one species has almost categorically longer petals

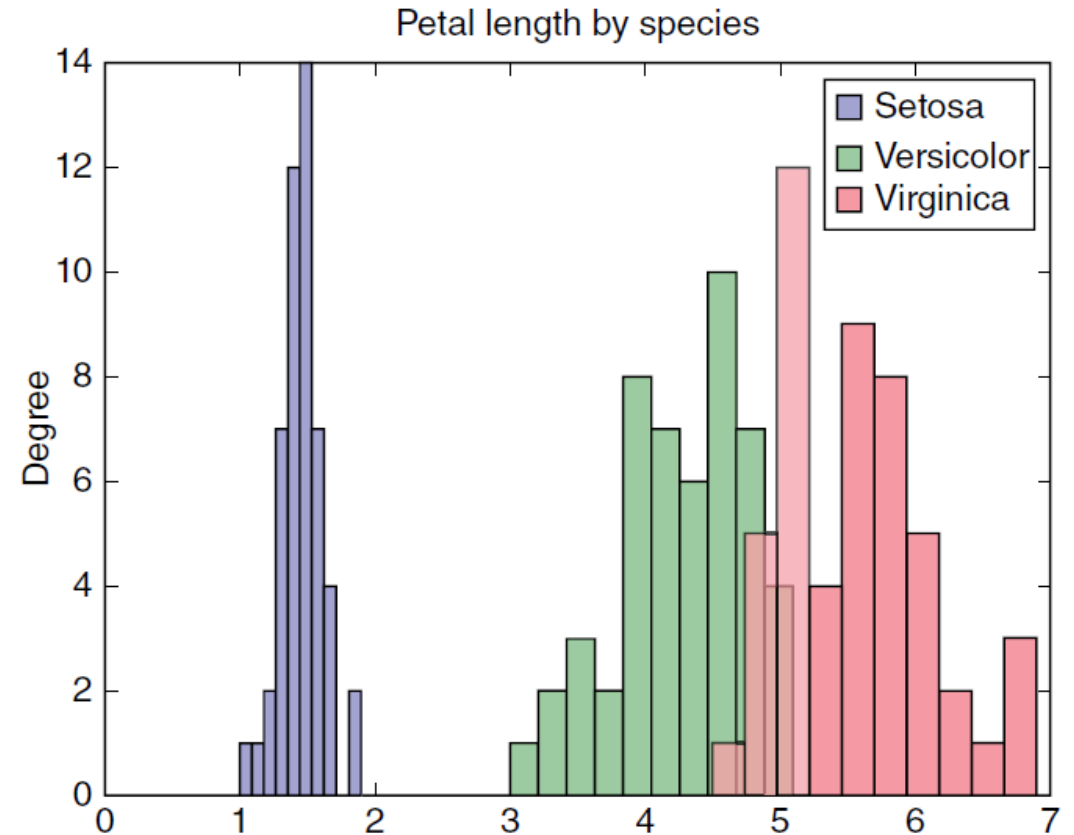
Histograms

- a sense of whether there are a few distinct outliers
- how much variation is in the population etc.

#plotting each species separately, but on the same axes and in different colors:

```
for spec in df['species'].unique():
    forspec = df[df['species']==spec]
    forspec['petal length (cm)'].plot(
        kind='hist', alpha=0.4, label=spec)

plt.legend(loc='upper right')
plt.suptitle('Petal Length by Species')
plt.savefig('iris_hist_by_spec.jpg')
```



- petal length has a clearly bimodal distribution, suggesting that this one species has almost categorically longer petals
- the peak on the left is only the iris setosa variety

Means, Standard Deviations, Medians, and Quantiles

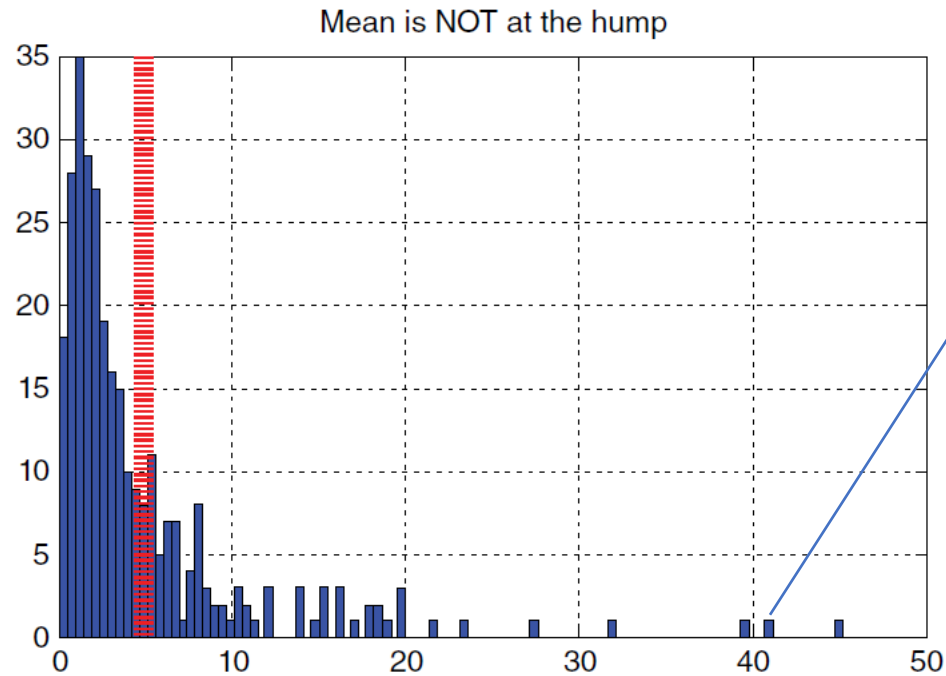
- summarize a distribution down to just a few meaningful numbers
- based on the assumption that your data's distribution is bell-shaped, and your goal is to give some idea of where the peak of the bell is and how widely it spreads.

- 1) Give the mean and standard deviation. These are the more historically popular metrics, and they are much easier to compute.
- 2) Give the median, 25th percentile, and 75th percentile. These metrics are more robust to pathologies in the data, but they are computationally more expensive.

```
col = df['petal length (cm)']  
Average = col.mean()  
Std = col.std()  
Median = col.quantile(0.5)  
Percentile25 = col.quantile(0.25)  
Percentile75 = col.quantile(0.75)
```

Means, Standard Deviations, Medians, and Quantiles

Problem: multiple peaks, outliers



The handful of very large outliers have pulled the mean well to the right of the actual hump in the distribution.

For a real-world distribution, there always *is* a mean.

But when there are massive outliers, just a single data point is likely to dominate the value of the mean and standard deviation.

Means, Standard Deviations, Medians, and Quantiles

A common solution is to, before calculating the mean, throw out all data points that are deemed to be outliers: a common criterion is anything below the 25th percentile or above the 75th percentile.

```
col = df['petal length (cm)']  
Perc25 = col.quantile(0.25)  
Perc75 = col.quantile(0.75)  
Clean_Avg = col[(col>Perc25)&(col<Perc75)].mean()
```

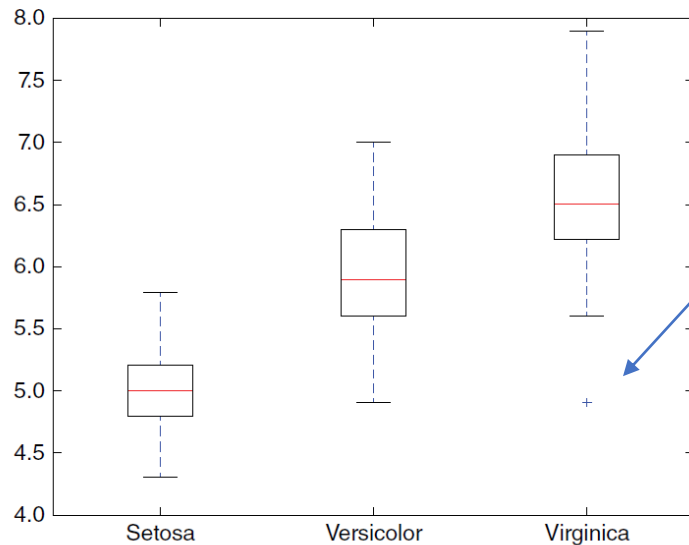
- For some cases though, the outliers are extremely important:
the amount of money in a transaction -> for fraud detection

Boxplots

- Boxplots are a convenient way to summarize a dataset by showing the median, quantiles, and min/max values for each of the variables.

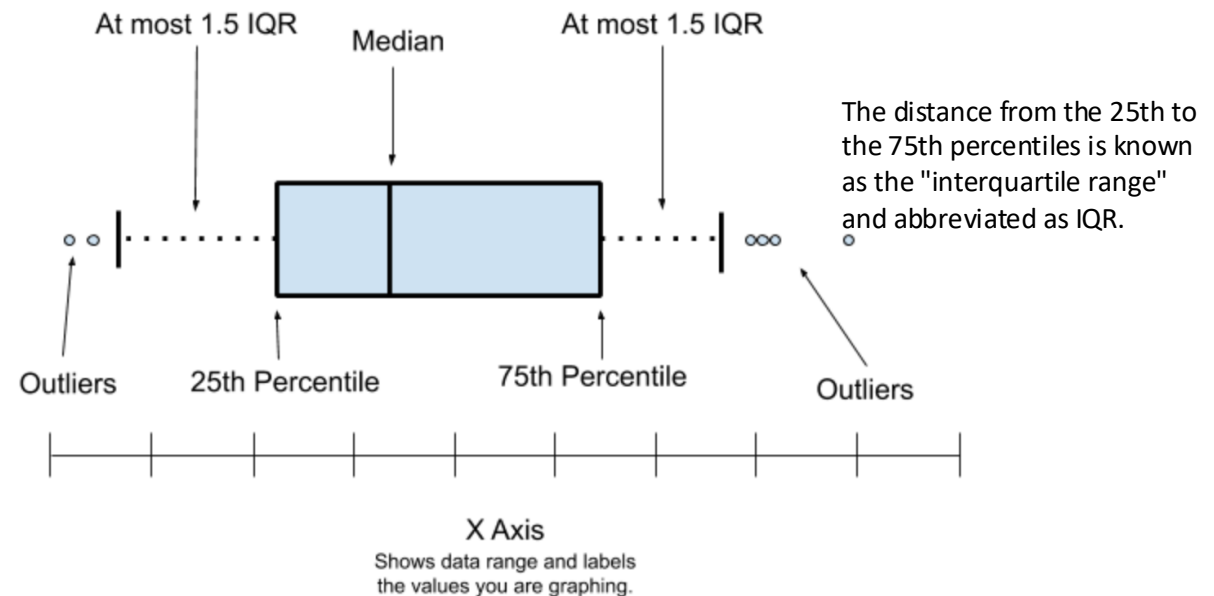
#boxplot of the sepal length for each of the species in the iris dataset:

```
col = 'sepal length (cm)'  
df['ind'] = pd.Series(df.index).apply(lambda i: i% 50)  
df.pivot('ind', 'species') [col].plot(kind='box')  
plt.show()
```



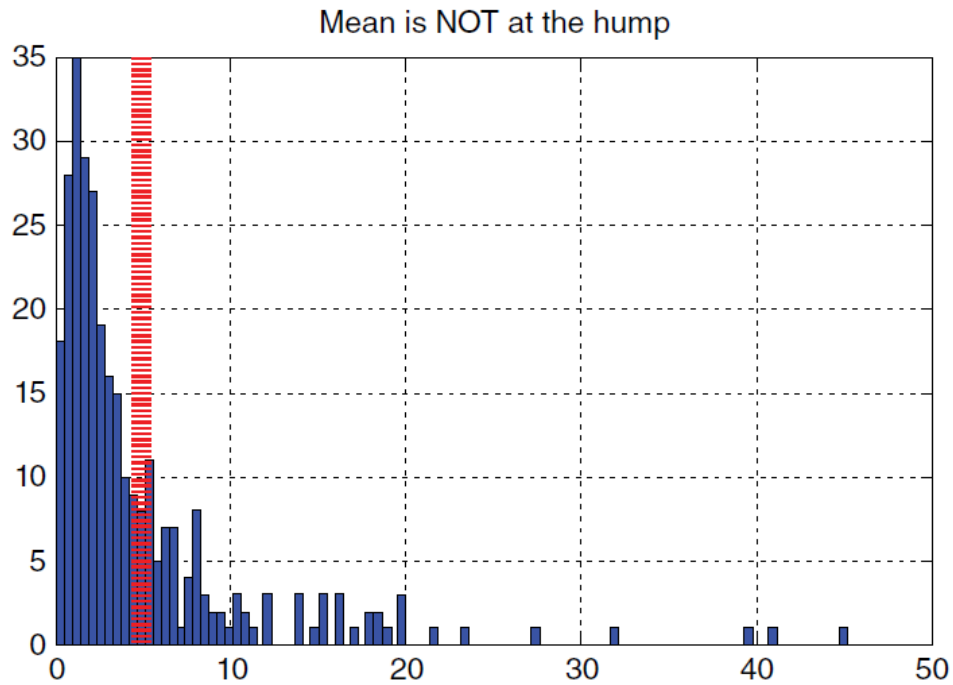
The boxplot makes it obvious that the three species are different from one another.

An advantage of boxplots is that major outliers are very visually obvious

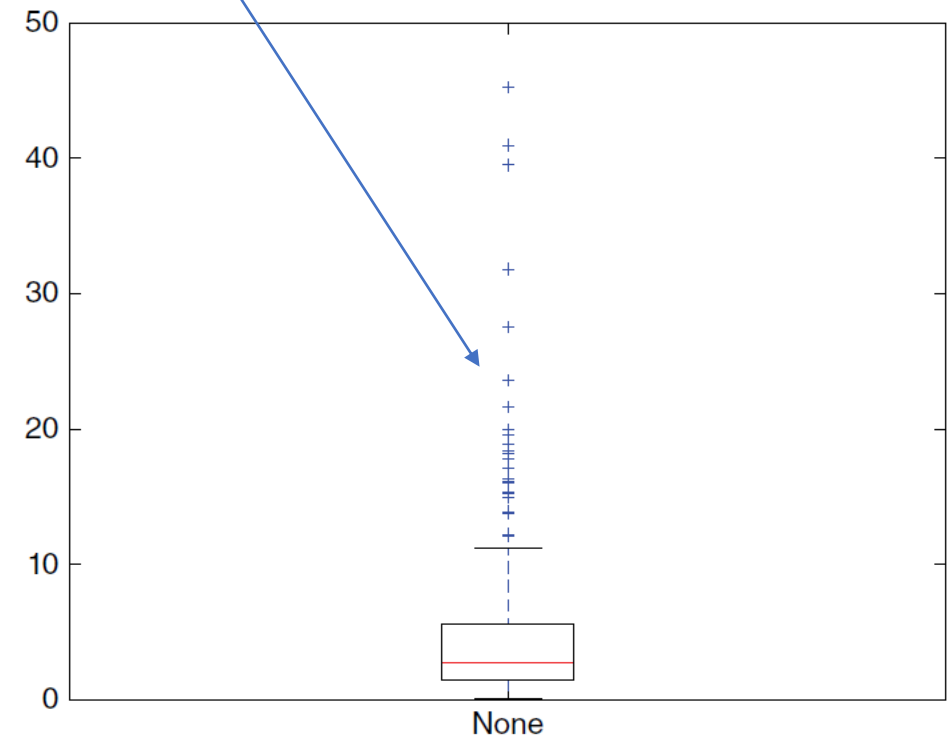


Boxplots

- Boxplots are a convenient way to summarize a dataset by showing the median, quantiles, and min/max values for each of the variables.



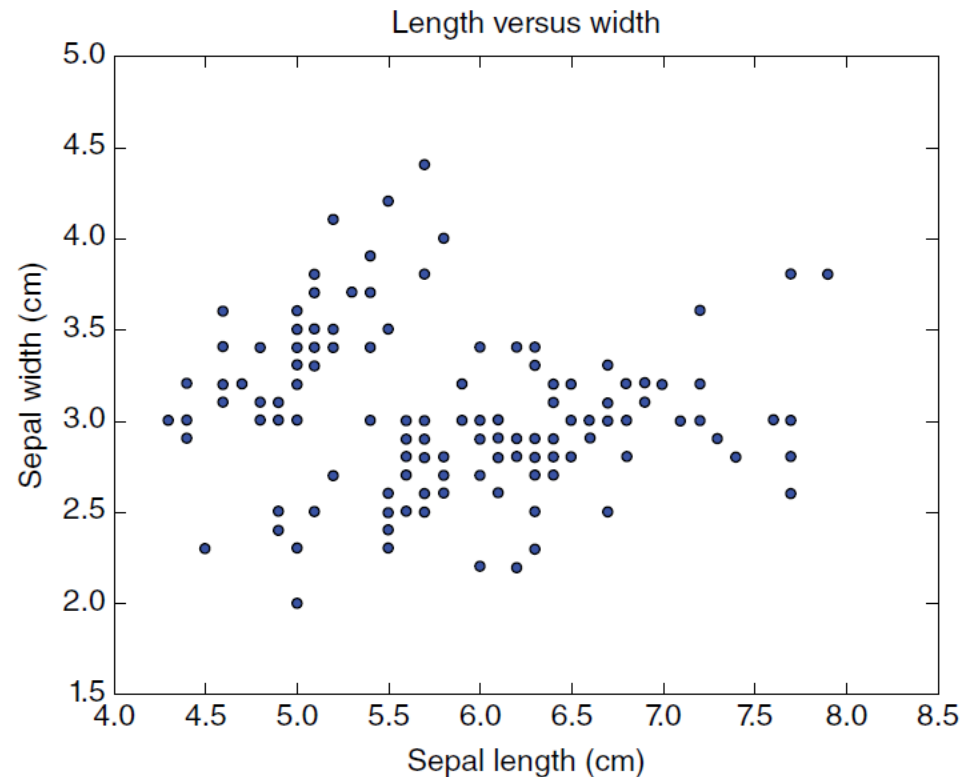
An advantage of boxplots is that major outliers are very visually obvious



Scatterplots

- Simple and powerful ways to visualize relationships within a dataset

```
df.plot(kind="scatter",  
        x="sepal length (cm)", y="sepal width (cm)")  
plt.title("Length vs Width")  
plt.show()
```



Scatterplots

Tips for better visualization:

- Color coding. Often data points that fall into different categories are given different colors.
- Size. Changing the size of data points communicates another dimension of information, similarly to color coding.
- Opacity. In scatterplots and other visualizations, it is often useful to make things partially transparent in case they overlap with other parts of the visualization.

By passing optional arguments into the `scatter()` function:

`c`: A string indicating the color to make the dots. You can also pass in a sequence of such strings if the dots are to be of different colors.

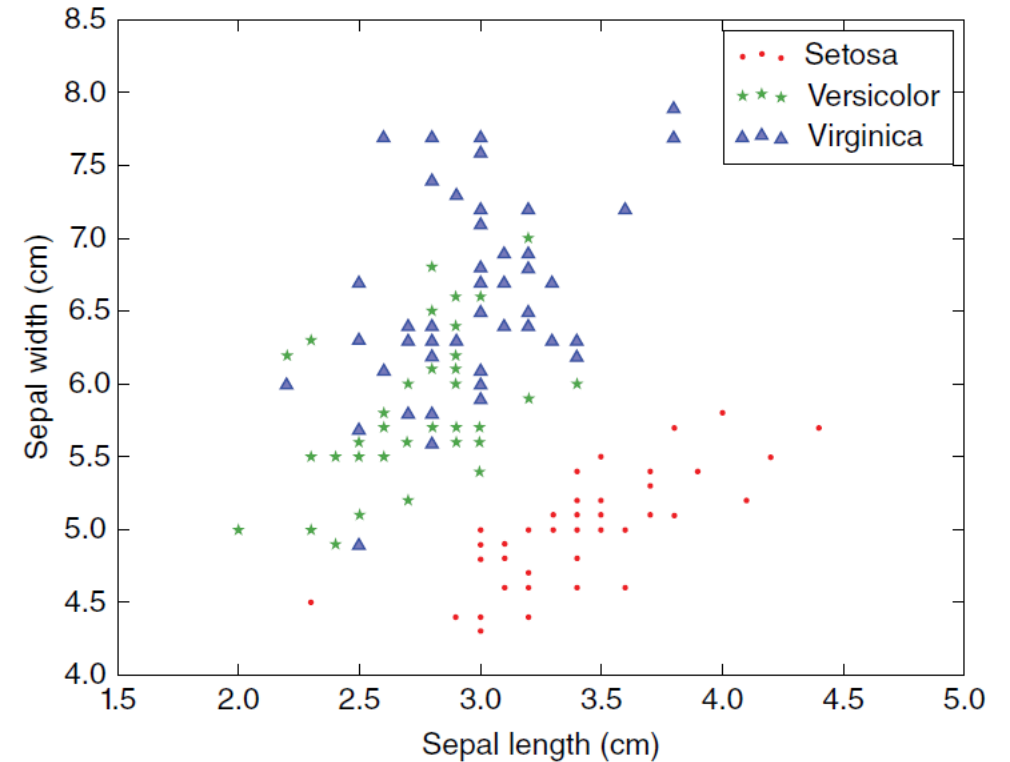
`s`: The size that each point should be, in pixels. Alternatively, you can pass in a sequence of sizes.

`marker`: A string indicating what marker should be used in the plot.

`alpha`: The transparency.

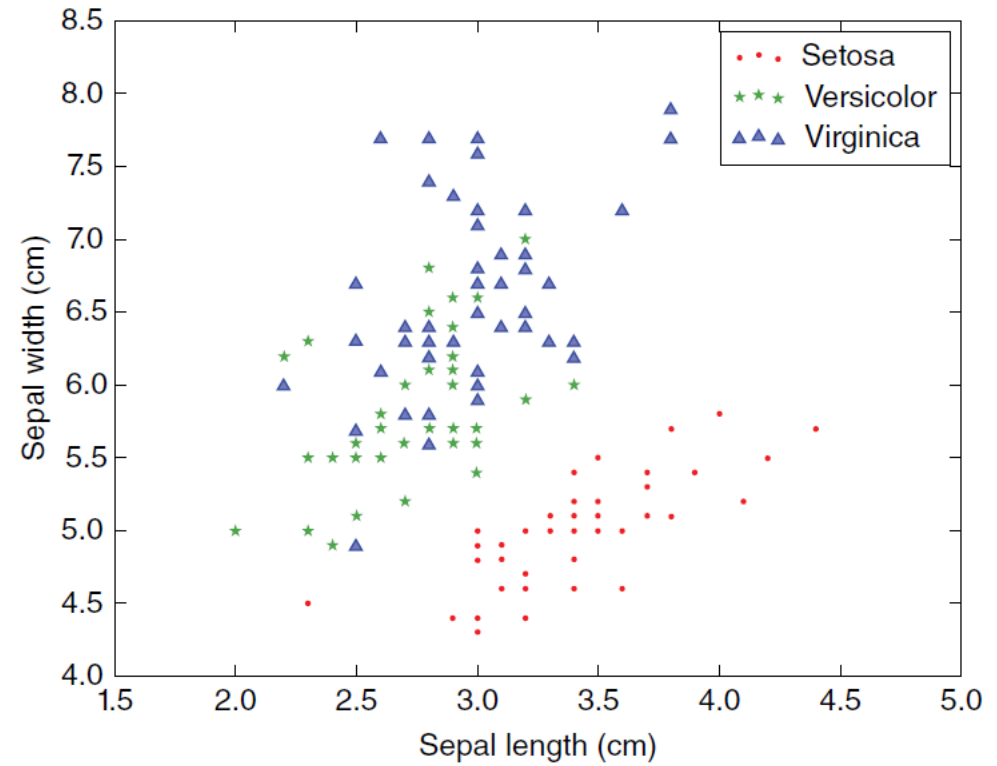
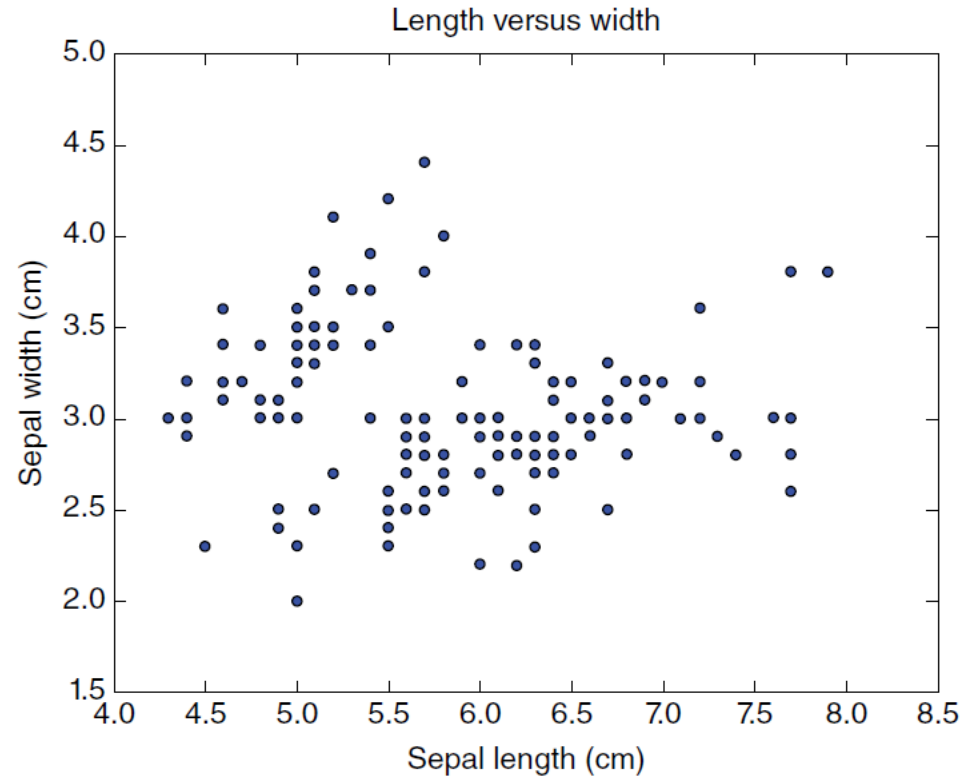
Scatterplots

```
plt.close()
colors = ["r", "g", "b"]
markers= [".", "*", "^"]
fig, ax = plt.subplots(1, 1)
for i, spec in enumerate(df['species'].unique() ):
    ddf = df[df['species']==spec]
    ddf.plot(kind="scatter",
            x="sepal width (cm)", y="sepal length (cm)",
            alpha=0.5, s=10*(i+1), ax=ax,
            color=colors[i], marker=markers[i], label=spec)
plt.legend()
plt.show()
```



- Iris setosa flowers stand apart as having sepals that are markedly longer and narrower.

Scatterplots

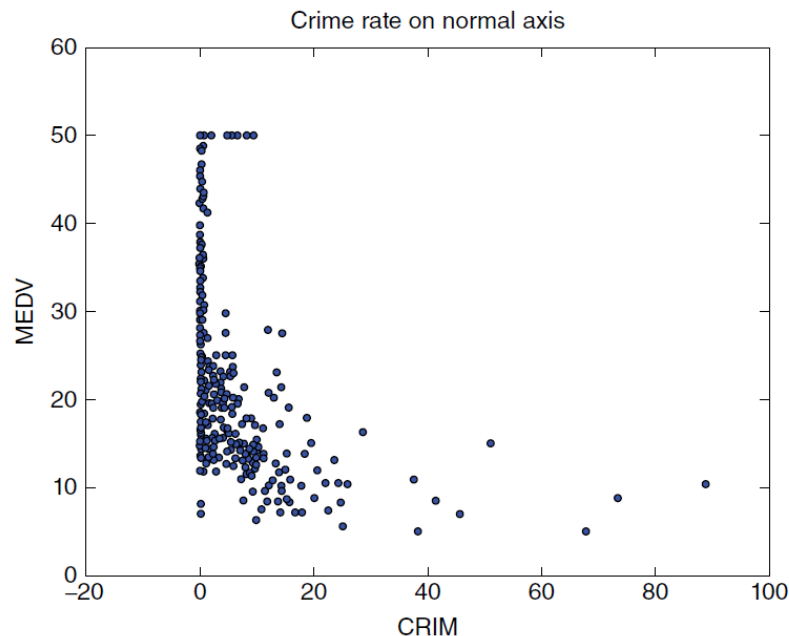


Scatterplots with Logarithmic Axes

A key variation on scatterplots is using logarithmic axes. In many applications, the numbers being plotted are all positive (or at least nonnegative), but they can vary by orders of magnitude.

In a scatterplot of data such as this, all but the largest data points will be squashed to one side, making the plot essentially unreadable.

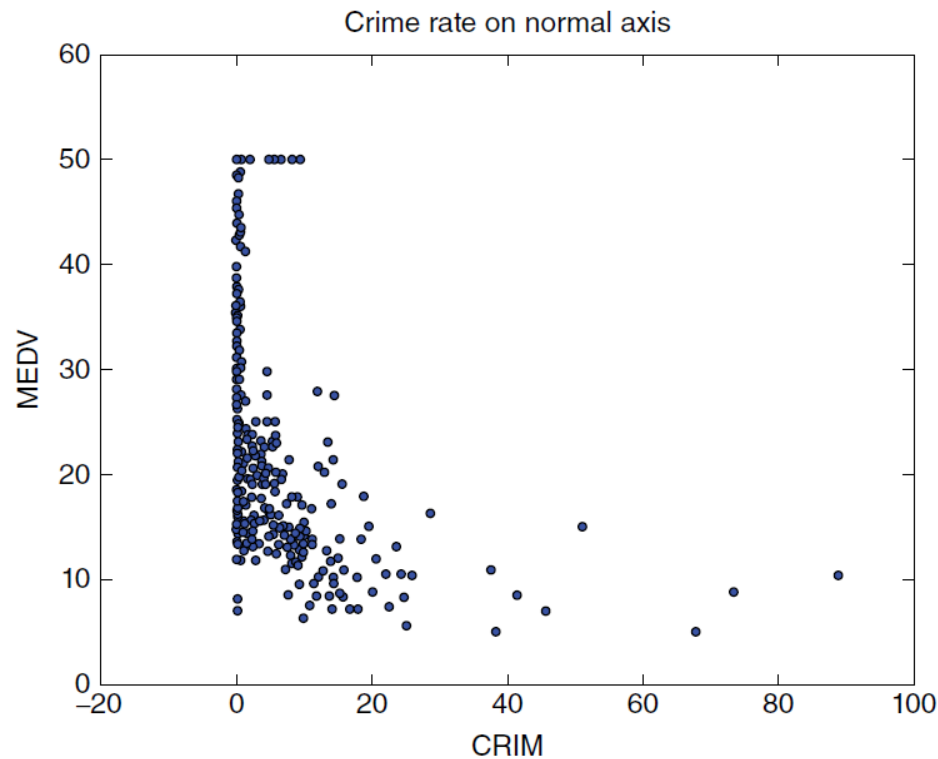
A scatterplot of the crime rate in a neighborhood versus the median home value:



Note how almost all of the data points are squashed to the left, making the graph hard to read for all but the most high-crime neighborhoods.

Scatterplots with Logarithmic Axes

Instead, we could have made the x-axis logarithmic:

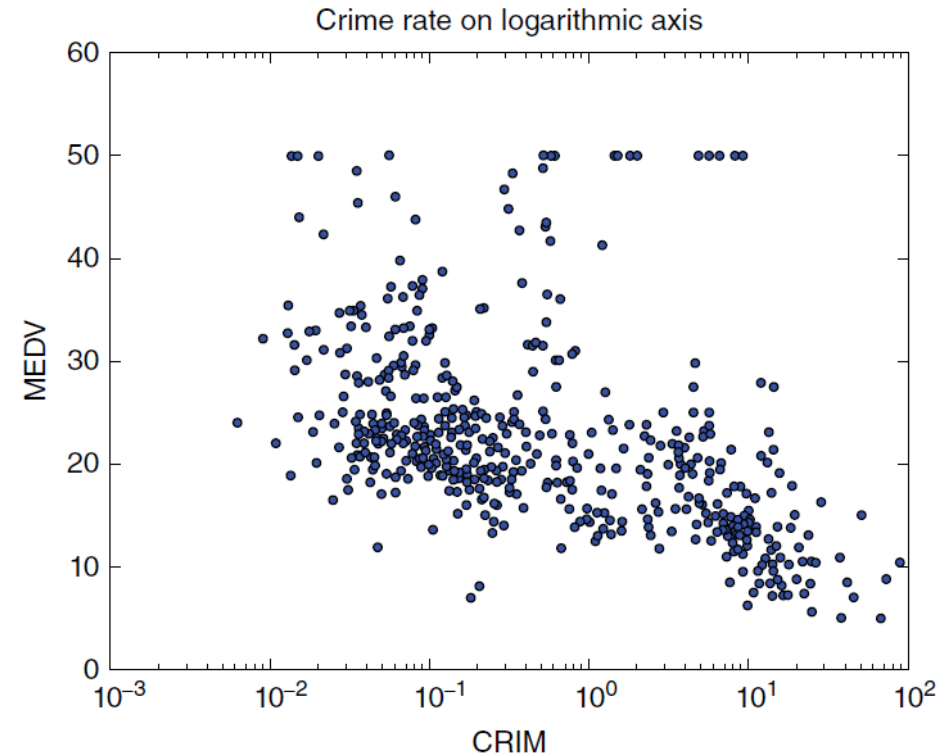


Note how almost all of the data points are squashed to the left, making the graph hard to read for all but the most high-crime neighborhoods.

Ref: The Data Science Handbook by Field Cady, Wiley

We make a logarithmic plot by taking the log of the raw data and using that to tell where to place the points.

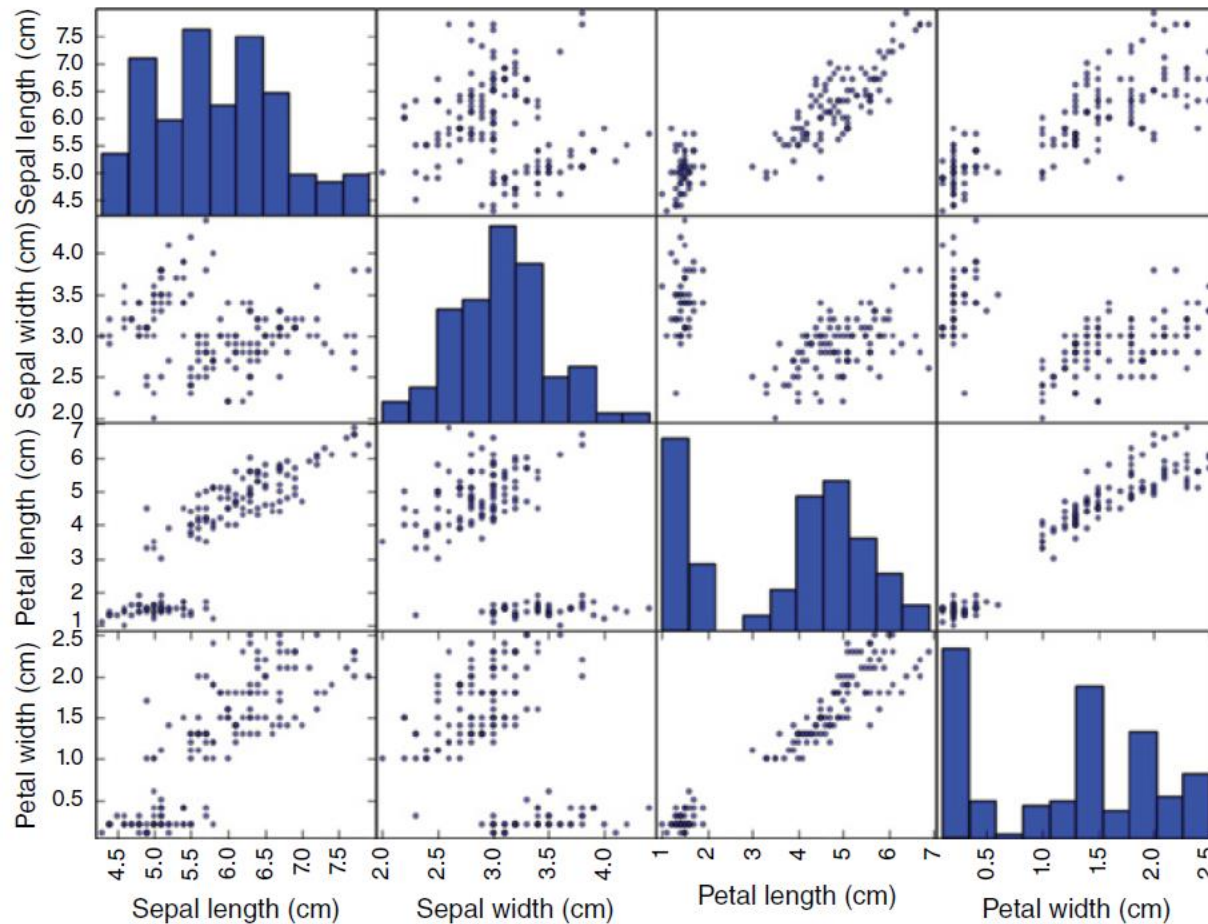
- Logarithmic plots only work when all values are greater than 0.



With this rescaling, we can see that there is a clear inverse relationship between crime rate and median home value that exists across all levels of crime. Taking the log of the raw data ensures that spacing between points corresponds correctly to orders of **magnitude**.

Scatter Matrices

scatterplot comparing every pair of features, arranging them in what's sometimes called a “scatter matrix”



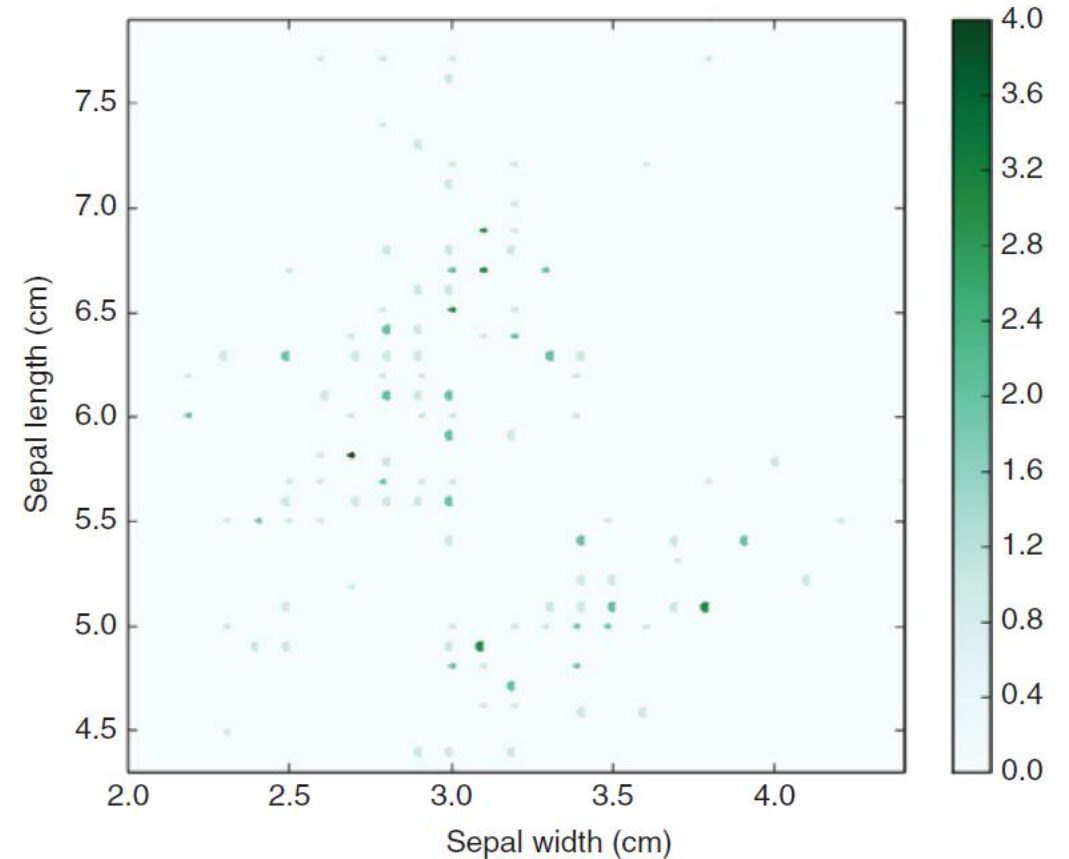
Note that along the diagonal, we have a histogram of each feature, rather than a scatterplot of the feature versus itself (which would just be a straight line).

Heatmaps

scatterplots is that they can become visually cluttered if you have a lot of data points.

- You may reduce the size of the data points
- alpha parameter to adjust the transparency of the points

Or: we don't actually care about the actual points themselves. We care about the density of points in the different regions. -> we can visualize this with a heatmap which color-codes different regions of the plane by their relative density of points.



Time Series

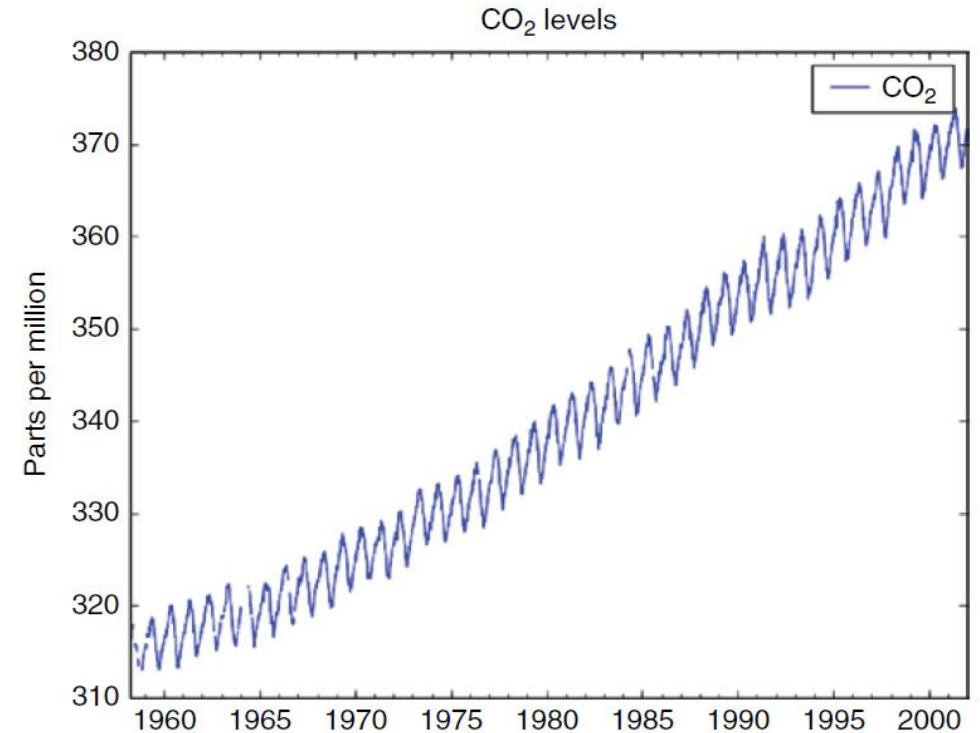
Time series data is one of the most important data types in the world, capturing everything from stock prices, to website traffic, to blood glucose levels.

- There could be noisy spikes that have to be filtered out, weird plateaus.
- Means and standard deviations are based on the naïve assumption that data follows pretty bell curves, but there is no corresponding “default” assumption for time series data.

You might want to

- zoom in on a sharp spike
- immediate step downs can be exponential decay
- to plot not just the data itself but the log of the data
- Plot multiple time series on top of each other to see patterns
- look at transformations of data into the frequency domain
 - expectation of periodicity
 - Fourier transform

Measurements of atmospheric CO₂ levels over many years



CO₂ fluctuates on a yearly cycle and
It increases overall across time.

#Todo

- Try out the examples yourself! The code is included, start from the examples here and explore more
- Creative Data Visualization
- Check data Sculptures, architecture by Refik Anadol:
 - https://www.ted.com/talks/refik_anadol_art_in_the_age_of_machine_intelligence?language=tr#t-422183 (with subtitles)
 - <https://www.wired.com/story/artist-refik-anadol-turns-data-art-help-ai/>

Frank Gehry's Walt Disney Concert Hall,

Todo:

- Learning Resources: Week 1 (lec 2/2) Data Quality
- Read Paper: Tasks and Visualizations Used for Data Profiling:
A Survey and Interview Study

A Practical Guide to Data Quality and Data Profiling

Watch:

Visualizing the Quality of Data (video)

Vizdataquality (Python package)

Todo:

- Learning Resources: Week 2 EDA and Viz
- Tutorials:
 - recreating_gapminder_viz.ipynb
 - EDA Case Study: Tuberculosis in the US

References:

- Pavlos Protopapas, Kevin Rader and Chris Tanner (data)
- Joseph E. Gonzalez
- Narges Norouzi